

# Monter un lab de pentest sur une VPS dédiée

Provisioning, durcissement, isolation réseau, déploiement de cibles vulnérables, workflow d'exploitation et intégration de l'IA — de zéro à un environnement d'entraînement opérationnel.

USAGE STRICTEMENT PÉDAGOGIQUE — SUR INFRASTRUCTURE PROPRE

# Sommaire

1. Objectifs & cadre légal
2. Choix et dimensionnement de la VPS dédiée
3. Provisioning initial & premier accès
4. Durcissement (hardening) de la VPS
5. Installation de Docker & Docker Compose
6. Architecture logique du lab
7. Déploiement des cibles web vulnérables
8. Ajout d'une cible réseau & de CVE réels (Vulhub)
9. La machine d'attaque & le toolset
10. Accès sécurisé : tunnels SSH & pivot SOCKS
11. Méthodologie d'exploitation
12. Intégrer l'IA dans le workflow
13. Snapshots, reset & maintenance
14. Checklist de sécurité du lab
15. Parcours d'apprentissage & ressources

# 1. Objectifs & cadre légal

---

Un lab offensif sert à acquérir l'intuition qui distingue un praticien d'un exécutant de scripts : reconnaître un pattern vulnérable, comprendre *pourquoi* une faille existe, et savoir l'exploiter proprement.

## Ce que ce lab permet

- Pratiquer l'ensemble de la kill chain web et réseau sur des cibles conçues pour être cassées.
- Étudier des CVE réels (Log4Shell, Struts, désérialisations...) dans des environnements reproductibles.
- Construire un workflow professionnel : reconnaissance → énumération → exploitation → post-exploitation → reporting.
- Boucler les deux facettes de ton profil : déployer et sécuriser (DevOps), puis attaquer (offensif).

**Cadre légal — non négociable.** Tu n'attaques que ce qui t'appartient ou ce pour quoi tu détiens une autorisation écrite. Ce lab, les plateformes légales (PortSwigger, HTB, THM) et tes propres machines sont faits pour ça. Tester un système tiers sans mandat est un délit dans la quasi-totalité des juridictions, Sénégal inclus (loi n°2008-11 sur la cybercriminalité). La frontière entre un professionnel et un problème judiciaire, c'est l'autorisation.

**Principe directeur.** L'IA accélère le travail, elle ne remplace pas le jugement. Tu montes le lab pour garder le muscle manuel *et* apprendre à piloter l'IA comme multiplicateur — pas pour copier-coller des payloads que tu ne comprends pas.

## 2. Choix et dimensionnement de la VPS dédiée

Une VPS **séparée de ta VPS de dev** est le bon choix : tu y héberges des applis volontairement vulnérables, tu dois pouvoir la casser, la réinstaller et la considérer comme non fiable sans jamais mettre en risque ton code ou tes clés de production.

### Dimensionnement recommandé

| Profil      | vCPU | RAM  | Disque    | Usage                                     |
|-------------|------|------|-----------|-------------------------------------------|
| Minimal     | 1    | 2 Go | 25 Go     | 2–3 cibles web (Juice Shop, DVWA)         |
| Confortable | 2    | 4 Go | 50 Go     | Cibles web + Vulhub + conteneur attaquant |
| Aisé        | 2–4  | 8 Go | 80 Go SSD | Lab multi-cibles, snapshots fréquents     |

### Critères de choix du fournisseur

- **Snapshots à la demande** : indispensable pour réinitialiser le lab en un clic.
- **Facturation horaire** : tu détruis la VPS quand tu ne t'en sers pas.
- **API** : provisioning reproductible (Terraform, CLI).
- Fournisseurs adaptés : Hetzner (excellent rapport prix/perf), DigitalOcean, Vultr, Scaleway, Contabo. Compte 4–6 €/mois pour le profil confortable.

**Astuce.** Prends une image **Debian 12** ou **Ubuntu 24.04 LTS** : bases les plus documentées pour Docker et le tooling de sécurité.

## 3. Provisioning initial & premier accès

---

### 3.1 Générer une paire de clés SSH dédiée (sur ton laptop)

N'utilise jamais de mot de passe pour te connecter. Génère une clé Ed25519 réservée au lab :

```
# Sur TON laptop, pas sur la VPS
ssh-keygen -t ed25519 -a 100 -C "lab-vps" -f ~/.ssh/lab_vps
```

Lors de la création de la VPS, colle le contenu de `~/.ssh/lab_vps.pub` dans le champ « clé SSH » du fournisseur.

### 3.2 Première connexion

```
ssh -i ~/.ssh/lab_vps root@<IP_DE_LA_VPS>
```

### 3.3 Alias de connexion (confort)

Ajoute dans `~/.ssh/config` sur ton laptop :

```
Host lab
  HostName <IP_DE_LA_VPS>
  User bamba
  IdentityFile ~/.ssh/lab_vps
  IdentitiesOnly yes
```

Tu te connecteras ensuite simplement avec `ssh lab`.

## 4. Durcissement (hardening) de la VPS

---

Même dédiée au lab, la VPS est exposée sur Internet par son port SSH. On réduit la surface d'attaque au strict minimum : SSH par clé uniquement, pare-feu en *deny by default*, bannissement des bruteforce, mises à jour automatiques.

### 4.1 Mises à jour & utilisateur non-root

```
apt update && apt -y full-upgrade

# Créer un utilisateur d'administration
adduser bamba
usermod -aG sudo bamba

# Copier la clé SSH vers le nouvel utilisateur
rsync --archive --chown=bamba:bamba ~/.ssh /home/bamba
```

### 4.2 Verrouiller SSH

Édite `/etc/ssh/sshd_config` (ou un fichier dans `/etc/ssh/sshd_config.d/`) :

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
X11Forwarding no
MaxAuthTries 3
AllowUsers bamba
```

Applique et vérifie **sans fermer ta session actuelle** (garde un second terminal ouvert en secours) :

```
sshd -t # valide la syntaxe
systemctl restart ssh
```

**Piège classique.** Teste toujours une nouvelle connexion SSH dans un *nouveau* terminal avant de fermer la session en cours. Une erreur dans `sshd_config` peut te verrouiller dehors définitivement.

### 4.3 Pare-feu UFW — deny by default

```
apt -y install ufw
ufw default deny incoming
ufw default allow outgoing
ufw allow OpenSSH
ufw enable
ufw status verbose
```

**Point clé de l'architecture.** On n'ouvre **aucun** port des cibles vulnérables sur Internet. UFW ne laisse entrer que SSH. Tout l'accès aux applis vulnérables passera par tunnel SSH (section 10).

## 4.4 fail2ban — bannir les bruteforce SSH

```
apt -y install fail2ban
cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

Dans `/etc/fail2ban/jail.local`, active la jail SSH :

```
[sshd]
enabled = true
maxretry = 4
bantime = 1h
findtime = 10m
```

```
systemctl enable --now fail2ban
fail2ban-client status sshd
```

## 4.5 Mises à jour de sécurité automatiques

```
apt -y install unattended-upgrades
dpkg-reconfigure -pnow unattended-upgrades
```

**Attention Docker & UFW.** Docker écrit directement dans iptables et peut *contourner* les règles UFW en publiant des ports. C'est précisément pour cela qu'on bindera tous les ports des cibles sur `127.0.0.1` (voir section 7) : un port publié sur `127.0.0.1` n'est jamais joignable depuis l'extérieur, quel que soit l'état d'UFW.

## 5. Installation de Docker & Docker Compose

---

Docker est le socle du lab : chaque cible est un conteneur jetable, isolé, reconstruisible en une commande.

### 5.1 Installation depuis le dépôt officiel

```
# Dépendances
apt -y install ca-certificates curl gnupg

# Clé GPG officielle Docker
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | \
  gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

# Dépôt (remplacer 'debian' par 'ubuntu' si Ubuntu)
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/debian $(. /etc/os-release && echo $VERSION_CODENAME)
stable" \
  | tee /etc/apt/sources.list.d/docker.list

apt update
apt -y install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

### 5.2 Autoriser ton utilisateur à piloter Docker

```
usermod -aG docker bamba
# Déconnecte-toi / reconnecte-toi pour appliquer
docker run --rm hello-world # test
docker compose version
```

**Rappel sécurité.** Appartenir au groupe **docker** équivaut à un accès root sur l'hôte. Acceptable sur une VPS dédiée et jetable ; à ne jamais faire sur ta VPS de dev/prod.

## 6. Architecture logique du lab

Deux modèles selon ce que tu veux pratiquer. Tu peux commencer par le A et ajouter le B ensuite.

### Modèle A — Web, piloté depuis le laptop

Les cibles tournent sur la VPS, bindées sur `127.0.0.1`. Tu attaques depuis ton laptop (Burp, ffuf, sqlmap) à travers un tunnel SSH. Simple, idéal pour l'offensif web.

```
[ Ton laptop ]                [ VPS dédiée ]
Burp / ffuf / sqlmap  --- SSH -L ---> 127.0.0.1:3000  juice-shop
navigateur                127.0.0.1:8081  dvwa
                            127.0.0.1:8080  webgoat
                            (rien n'est exposé sur l'IP publique – UFW ne laisse passer que SSH)
```

### Modèle B — Attaquant embarqué (réseau, Metasploit, pivot)

Un conteneur attaquant (Kali) tourne sur la VPS, dans le *même réseau Docker* que les cibles. Le trafic d'exploitation reste interne ; tu pilotes l'attaquant en SSH ou via un shell web. Idéal pour l'exploitation réseau et l'entraînement au pivot.

```
[ Ton laptop ] --- SSH ---> [ VPS ]
                             └─ réseau Docker "lab" (172.30.0.0/24)
                                └─ attacker (kali)   172.30.0.10
                                └─ juice-shop        172.30.0.20
                                └─ dvwa              172.30.0.21
                                └─ metasploitable    172.30.0.30
```

**Isolation réseau.** On place les cibles dans un réseau Docker dédié `lab`. Elles se voient entre elles et voient l'attaquant, mais l'exposition vers Internet reste bloquée par le binding `127.0.0.1` et UFW.

## 7. Déploiement des cibles web vulnérables

---

Crée un dossier de lab et le fichier Compose :

```
mkdir -p ~/lab && cd ~/lab
nano docker-compose.yml
```

### docker-compose.yml

```
services:
  # OWASP Juice Shop – SPA + API REST, référence du web offensif moderne
  juice-shop:
    image: bkimminich/juice-shop
    container_name: lab-juiceshop
    ports:
      - "127.0.0.1:3000:3000" # local uniquement
    networks: [lab]
    restart: unless-stopped

  # DVWA – difficulté réglable (low / medium / high / impossible)
  dvwa:
    image: ghcr.io/digininja/dvwa:latest
    container_name: lab-dvwa
    ports:
      - "127.0.0.1:8081:80"
    networks: [lab]
    restart: unless-stopped

  # WebGoat – parcours d'apprentissage guidé + WebWolf
  webgoat:
    image: webgoat/webgoat
    container_name: lab-webgoat
    ports:
      - "127.0.0.1:8080:8080"
      - "127.0.0.1:9090:9090"
    networks: [lab]
    restart: unless-stopped

networks:
  lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.0.0/24
```

### Lancer et vérifier

```
docker compose up -d
docker compose ps
docker compose logs -f juice-shop # suivre le démarrage
```

| Cible      | Port local | Accès (après tunnel)          | Focus                                      |
|------------|------------|-------------------------------|--------------------------------------------|
| Juice Shop | 3000       | http://localhost:3000         | XSS, injection, JWT, logique métier, API   |
| DVWA       | 8081       | http://localhost:8081         | SQLi, command injection, file upload, CSRF |
| WebGoat    | 8080       | http://localhost:8080/WebGoat | leçons guidées, désérialisation, JWT       |

DVWA : identifiants par défaut **admin / password** , puis clique « Create / Reset Database ». Règle la difficulté dans l'onglet *DVWA Security*.

## 8. Ajout d'une cible réseau & de CVE réels

### 8.1 Metasploitable — cible réseau multi-services

Pour l'exploitation réseau (services vulnérables : FTP, SMB, SSH, bases de données), ajoute au `docker-compose.yml` :

```
metasploitable:
  image: tleemcjr/metasploitable2
  container_name: lab-metasploitable
  networks: [lab]
  # Pas de 'ports:' – accessible seulement depuis le réseau 'lab'
  # (donc depuis le conteneur attaquant, jamais depuis Internet)
```

**Ne publie jamais les ports de Metasploitable.** C'est une cible délibérément béante (services non authentifiés, backdoors). Elle ne doit exister que dans le réseau `lab`, atteignable uniquement par ton attaquant embarqué (Modèle B).

### 8.2 Vulhub — étudier de vrais CVE

Vulhub fournit, par CVE, un environnement Docker reproductible. C'est l'outil idéal pour comprendre une faille en profondeur.

```
git clone https://github.com/vulhub/vulhub.git ~/vulhub
cd ~/vulhub/log4j/CVE-2021-44228 # exemple : Log4Shell
cat README.md                  # contexte + étapes
docker compose up -d
```

**Boucle d'apprentissage.** Monte le CVE → exploite-le manuellement → puis fais-toi expliquer par l'IA *pourquoi* la faille existe, ligne par ligne dans le code source vulnérable. Éteins toujours l'environnement après ( `docker compose down` ) pour libérer les ressources.

## 9. La machine d'attaque & le toolset

---

### 9.1 Option 1 — depuis ton laptop (recommandé pour le web)

Installe le nécessaire localement et attaque à travers le tunnel SSH :

- **Burp Suite Community** — proxy d'interception, le cœur du test web.
- **ffuf / gobuster** — fuzzing de répertoires et paramètres.
- **sqlmap** — exploitation automatisée d'injections SQL.
- **nmap** — scan de ports et fingerprinting.
- **nikto, whatweb** — reconnaissance web.

### 9.2 Option 2 — conteneur attaquant embarqué (pour le réseau)

Ajoute un Kali dans le réseau **lab** :

```
attacker:
  image: kalilinux/kali-rolling
  container_name: lab-attacker
  networks: [lab]
  tty: true
  stdin_open: true
  command: sleep infinity
```

Puis installe le toolset dans le conteneur :

```
docker exec -it lab-attacker bash
apt update && apt -y install kali-linux-headless
# ou un set ciblé : nmap metasploit-framework hydra sqlmap ...
```

Depuis ce conteneur, les cibles sont joignables par leur nom : **ping juice-shop**, **nmap metasploitable**.

## 10. Accès sécurisé : tunnels SSH & pivot SOCKS

Aucun port de cible n'étant exposé, l'accès passe par SSH. Deux techniques : le forward local (port par port) et le proxy SOCKS dynamique (accès à tout le réseau interne).

### 10.1 Forward local — accéder aux cibles web

Depuis ton laptop, ouvre un tunnel qui mappe les ports distants vers ton localhost :

```
ssh -L 3000:localhost:3000 \  
-L 8081:localhost:8081 \  
-L 8080:localhost:8080 \  
-L 9090:localhost:9090 lab
```

Tunnel actif, ouvre `http://localhost:3000` dans ton navigateur : tu attaques Juice Shop comme si elle était locale. Configure Burp en proxy et navigue à travers.

### 10.2 Proxy SOCKS dynamique — pivot dans le réseau interne

Pour atteindre *n'importe quelle* IP du réseau Docker (ex. Metasploitable en `172.30.0.30`) sans publier de port :

```
ssh -D 1080 lab # ouvre un proxy SOCKS5 sur localhost:1080
```

Configure **proxychains** sur ton laptop — dans `/etc/proxychains4.conf` :

```
[ProxyList]  
socks5 127.0.0.1 1080
```

Puis fais transiter tes outils par le tunnel :

```
proxychains nmap -sT -Pn 172.30.0.30  
proxychains curl http://172.30.0.20:3000
```

**Pourquoi -sT.** À travers un proxy SOCKS, seuls les scans TCP *connect* ( `-sT` ) fonctionnent : le SYN-scan bas niveau ne traverse pas le proxy. Ajoute `-Pn` pour désactiver la découverte d'hôte.

# 11. Méthodologie d'exploitation

---

Un déroulé professionnel reproductible, du premier contact au rapport.

## Phase 1 — Reconnaissance & énumération

```
# Ports et services
nmap -sV -sC -p- -oN scan.txt 172.30.0.20

# Découverte de contenu web
ffuf -w wordlist.txt -u http://localhost:3000/FUZZ -mc 200,301,302
whatweb http://localhost:3000
```

## Phase 2 — Analyse des vulnérabilités

- Cartographier la surface d'attaque : points d'entrée, paramètres, endpoints d'API, upload, authentification.
- Intercepter et rejouer les requêtes dans Burp Repeater ; observer les écarts de comportement.
- Confronter les versions détectées à une base de CVE.

## Phase 3 — Exploitation

```
# Injection SQL
sqlmap -u "http://localhost:8081/vulnerabilities/sqli?id=1" \
  --cookie="PHPSESSID=...; security=low" --batch --dbs

# Bruteforce d'authentification
hydra -l admin -P rockyou.txt localhost -s 8081 http-post-form \
  "/login.php:username=^USER^&password=^PASS^:Login failed"
```

**Discipline.** Ne lance jamais un exploit — surtout généré par une IA — sans l'avoir lu et compris. Un PoC mal maîtrisé peut détruire la cible (ou pire, échapper au lab). Comprendre avant d'exécuter est la règle.

## Phase 4 — Post-exploitation

- Établir une session stable (reverse shell → shell interactif).
- Énumération locale : utilisateurs, droits, tâches planifiées, secrets, chemins d'escalade de privilèges.
- Documenter chaque étape au fil de l'eau (captures, commandes, sorties).

## Phase 5 — Reporting

Pour chaque vulnérabilité : description, impact, preuve d'exploitation reproductible, criticité (CVSS), remédiation. C'est la phase où l'IA fait gagner le plus de temps — sur une trame que tu contrôles.

## 12. Intégrer l'IA dans le workflow

L'IA change le rythme du travail, pas sa nature. Elle supprime les temps morts ; le jugement — où attaquer, pourquoi une faille existe, si un exploit est fiable — reste à toi.

### Là où l'IA est la plus rentable

| Usage           | Exemple concret                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| Tuteur          | « Explique-moi ce CVE ligne par ligne et le modèle mental derrière la technique. » |
| Mode socratique | « Donne-moi seulement le prochain angle d'attaque, pas la solution. »              |
| Parsing         | Corréler une sortie nmap/ffuf/Burp et prioriser les pistes.                        |
| Adaptation      | Ajuster un exploit public à la version exacte de la cible.                         |
| PoC & fuzzing   | Écrire un harnais de fuzzing ou un script d'exploitation jetable.                  |
| Reporting       | Structurer le rapport à partir de tes notes brutes.                                |

### Les garde-fous

- **Vérifier avant d'exécuter** : l'IA hallucine des payloads, des flags, des offsets. Relis systématiquement.
- **Entretenir l'intuition** : de temps en temps, coupe l'IA sur une machine et fais tout à l'ancienne. C'est ce muscle qui te permet de savoir quand l'IA se trompe.
- **Ne jamais déléguer le jugement** : l'IA propose, tu décides.

**Ton avantage.** Tu as fait le manuel d'abord. Quand l'IA te sort un exploit, tu sais le lire, le corriger, repérer ses erreurs. C'est exactement ce qui sépare un expert d'un dépendant de l'outil.

## 13. Snapshots, reset & maintenance

---

### 13.1 Réinitialiser une cible

```
# Repartir d'un état propre (efface volumes et données)
docker compose down -v
docker compose up -d
```

### 13.2 Snapshots au niveau VPS

Avant une session risquée (post-exploitation, malware d'entraînement), prends un snapshot chez ton fournisseur. En cas de compromission de l'hôte lui-même, restaure en quelques minutes.

### 13.3 Versionner la configuration

```
cd ~/lab
git init && git add docker-compose.yml && git commit -m "lab initial"
```

Ton lab devient reproductible : tu peux le détruire et le remonter à l'identique sur une nouvelle VPS.

### 13.4 Hygiène des ressources

```
docker compose stop      # éteindre sans détruire
docker system prune -f   # nettoyer images/conteneurs orphelins
docker stats              # surveiller la conso
```

## 14. Checklist de sécurité du lab

| #  | Contrôle                                  | État attendu              |
|----|-------------------------------------------|---------------------------|
| 1  | SSH par clé uniquement, root désactivé    | PasswordAuthentication no |
| 2  | UFW actif, deny incoming sauf SSH         | ufw status = active       |
| 3  | fail2ban actif sur sshd                   | jail sshd enabled         |
| 4  | Ports des cibles bindés sur 127.0.0.1     | aucun 0.0.0.0 exposé      |
| 5  | Metasploitable sans <b>ports:</b> publiés | réseau interne seul       |
| 6  | VPS distincte de la dev/prod              | isolation totale          |
| 7  | Snapshot récent disponible                | < dernière session        |
| 8  | Mises à jour automatiques actives         | unattended-upgrades       |
| 9  | Environnements Vulhub éteints après usage | docker compose down       |
| 10 | Aucune donnée réelle/sensible sur la VPS  | lab strictement jetable   |

**Vérification exposition.** Depuis un réseau externe (ton laptop, hors tunnel), exécute `nmap -sV <IP_VPS>` : seul le port 22 doit répondre. Tout autre port ouvert est une fuite à corriger immédiatement.

## 15. Parcours d'apprentissage & ressources

---

### Progression recommandée

1. **Socle** — HTTP en profondeur, internals Linux, réseau, bases de crypto.
2. **Web guidé** — PortSwigger Web Security Academy (gratuit, la meilleure ressource web) en parallèle de ton lab Juice Shop / DVWA.
3. **Machines** — TryHackMe (progressif) puis Hack The Box (mise en situation).
4. **CVE réels** — Vulhub, un CVE à la fois, exploité puis disséqué.
5. **Certification** — eJPT (entrée), puis PNPT ou OSCP / CPTS (validation d'un vrai niveau).

### Plateformes légales

| Ressource                        | Usage                            |
|----------------------------------|----------------------------------|
| PortSwigger Web Security Academy | Web offensif, gratuit, référence |
| TryHackMe                        | Parcours guidés progressifs      |
| Hack The Box                     | Machines et labs réalistes       |
| PentesterLab                     | Exercices web ciblés             |
| Vulhub                           | Environnements Docker par CVE    |
| VulnHub                          | VM vulnérables (usage local)     |

**Le mot de la fin.** Ton lab est une boucle : déploie, sécurise (DevOps), casse (offensif), documente, recommence. C'est en construisant *et* en cassant que l'intuition se forme — et c'est cette intuition, pas l'outil, qui fait l'expert.

---

Document pédagogique — sécurité offensive sur infrastructure propre. À n'appliquer que sur des systèmes t'appartenant ou pour lesquels tu détiens une autorisation écrite.